

RKNN3 TOOLKIT LITE User Guide

Document ID: RK-KF-YF-432

Version: V1.0.4

Date: 2026-05-06

Confidentiality: ☐Top Secret ☐Secret ☐Internal ☒Public

DISCLAIMER

THIS DOCUMENT IS PROVIDED "AS IS". ROCKCHIP ELECTRONICS CO., LTD. ("ROCKCHIP") DOES NOT PROVIDE ANY WARRANTY OF ANY KIND, EXPRESSED, IMPLIED OR OTHERWISE, WITH RESPECT TO THE ACCURACY, RELIABILITY, COMPLETENESS, MERCHANTABILITY, FITNESS FOR ANY PARTICULAR PURPOSE OR NON-INFRINGEMENT OF ANY REPRESENTATION, INFORMATION AND CONTENT IN THIS DOCUMENT. THIS DOCUMENT IS FOR REFERENCE ONLY. THIS DOCUMENT MAY BE UPDATED OR CHANGED WITHOUT ANY NOTICE AT ANY TIME DUE TO THE UPGRADES OF THE PRODUCT OR ANY OTHER REASONS.

Trademark Statement

"Rockchip", "瑞芯微", "瑞芯" shall be Rockchip's registered trademarks and owned by Rockchip. All the other trademarks or registered trademarks mentioned in this document shall be owned by their respective owners.

All rights reserved. ©2025. Rockchip Electronics Co., Ltd.

Beyond the scope of fair use, neither any entity nor individual shall extract, copy, or distribute this document in any form in whole or in part without the written approval of Rockchip.

Rockchip Electronics Co., Ltd.

No.18 Building, A District, No.89, software Boulevard Fuzhou, Fujian, PRC

Website: www.rock-chips.com

Customer service Tel: +86-4007-700-590

Customer service Fax: +86-591-83951833

Customer service e-Mail: fae@rock-chips.com

Preface

Overview

This document is the user guide for RKNN3 TOOLKIT LITE.

Target Audience

This document is primarily intended for the following engineers:

- Technical Support Engineers
- Software Development Engineers

Revision History

Version	Author	Date	Description	Approved by
V0.5.0	HPC Team	2025-12-05	Initial version	Vincent
V1.0.0	HPC Team	2026-01-05	Improve API documentation	Vincent
V1.0.4	HPC Team	2026-05-06	1. Added input callback 2. Added LLM inference pause and resume	Vincent

Table of Contents

RKNN3 TOOLKIT LITE User Guide

- 1 Overview
 - 1.1 Hardware Platforms
 - 1.2 Applicable system
- 2 Development environment build
 - 2.1 Requirements/Dependencies
 - 2.2 Installation
- 3 Usage
 - 3.1 Basic Usage Process
 - 3.2 Example
- 4 RKNN3 Type Definition
 - 4.1 Constants
 - 4.2 Enumerations
 - 4.2.1 RKNN3QueryCmd
 - 4.2.2 RKNN3TensorType
 - 4.2.3 RKNN3TensorQntType
 - 4.2.4 RKNN3TensorLayout
 - 4.2.5 RKNN3MemAllocFlags
 - 4.2.6 RKNN3MemSyncMode
 - 4.2.7 RKNN3CoreMask
 - 4.2.8 RKNN3KVCachePolicy
 - 4.2.9 RKNN3KVCacheClearPolicy
 - 4.2.10 RKNN3LLMInputType
 - 4.2.11 LLMCallState
 - 4.2.12 Other Enumerations
 - 4.3 Basic Structures
 - 4.3.1 RKNN3InputOutputNum
 - 4.3.2 RKNN3QuantInfo
 - 4.3.3 RKNN3TensorAttr
 - 4.3.4 RKNN3PerfDetail
 - 4.3.5 RKNN3PerfRun
 - 4.3.6 RKNN3SDKVersion
 - 4.3.7 RKNN3MemSize
 - 4.3.8 RKNN3CustomString
 - 4.3.9 RKNN3TensorMemory
 - 4.3.10 RKNN3Config
 - 4.3.11 RKNN3Tensor
 - 4.3.12 RKNN3AllocationInfo
 - 4.3.13 RKNN3ShapeInfo
 - 4.3.14 RKNN3ShapeConfig
 - 4.3.15 RKNN3InitExtend
 - 4.3.16 RKNN3NodeMemInfo
 - 4.3.17 RKNN3DevMemInfo
 - 4.3.18 RKNN3Device
 - 4.3.19 RKNN3Devices
 - 4.3.20 Other Structures
 - 4.4 LLM Related Structures
 - 4.4.1 Float16
 - 4.4.2 RKNN3VocabInfo
 - 4.4.3 RKNN3SamplingParams

- 4.4.4 RECURRENT
- 4.4.5 RKNN3KVCachePolicyParam
- 4.4.6 RKNN3Lora (Not currently supported)
- 4.4.7 RKNN3LLMTensor
- 4.4.8 RKNN3AuxTensor
- 4.4.9 RKNN3Image
- 4.4.10 RKNN3Audio
- 4.4.11 RKNN3Video
- 4.4.12 RKNN3LLMMultiModelTensor
- 4.4.13 RKNN3LLMInputUnion
- 4.4.14 RKNN3LLMInput
- 4.4.15 RKNN3LLMExtendParam
- 4.4.16 RKNN3LLMParam
- 4.4.17 RKNN3LLMInferParam
- 4.4.18 RKLLMResult
- 4.4.19 RKLLMRunState
- 4.4.20 RKLLMResultLastHiddenLayer
- 4.5 Callback Types
 - 4.5.1 LLMResultCallback
 - 4.5.2 LLMSamplingCallback
 - 4.5.3 LLMGetEmbedCallback
 - 4.5.4 LLMTokenizerCallback
 - 4.5.5 LLMOutputCallback
 - 4.5.6 LLMInputCallback
 - 4.5.7 RKLLMCallback
- 4.6 Wrapper Classes
 - 4.6.1 RKNN3AuxTensorWrapper
- 4.7 Utility Functions
 - 4.7.1 rknn_dtype_to_numpy_dtype
 - 4.7.2 rknn3_get_layout_string
 - 4.7.3 rknn3_get_type_string
 - 4.7.4 rknn3_get_qnt_type_string
 - 4.7.5 dump_tensor_attr
 - 4.7.6 get_os_platform
- 4.8 RKNN3Lite Class
 - 4.8.1 init
 - 4.8.2 load_rknn
 - 4.8.3 init_runtime
 - 4.8.4 inference
 - 4.8.5 session_run
 - 4.8.6 get_sdk_version
 - 4.8.7 set_chat_template
 - 4.8.8 get_devices_id
 - 4.8.9 get_inputs_tensor_attr
 - 4.8.10 get_outputs_tensor_attr
 - 4.8.11 init_lora (Not currently supported)
 - 4.8.12 load_lora (Not currently supported)
 - 4.8.13 unload_lora (Not currently supported)
 - 4.8.14 enable_lora (Not currently supported)
 - 4.8.15 disable_lora (Not currently supported)
 - 4.8.16 query_lora (Not currently supported)
 - 4.8.17 set_kvcache_policy
 - 4.8.18 load_kvcache

4.8.19 save_kvcache
4.8.20 clear_kvcache
4.8.21 session_run_async
4.8.22 session_stop
4.8.23 session_pause
4.8.24 session_resume
4.8.25 session_query_state
4.8.26 set_function_tools
4.8.27 set_llm_param
4.8.28 dup_context
4.8.29 inference_async
4.8.30 wait
4.8.31 get_outputs
4.8.32 create_mem
4.8.33 create_mem_from_phys
4.8.34 create_mem_from_fd
4.8.35 destroy_mem
4.8.36 mem_sync
4.8.37 set_shape
4.8.38 set_kvcache_mem
4.8.39 set_internal_mem
4.8.40 dump_features
4.8.41 profile_ops
4.8.42 profile_mem
4.8.43 register_custom_ops_plugins
4.8.44 im_mem_create
4.8.45 im_mem_destroy
4.8.46 im_cvt_color
4.8.47 create_output_tensors
4.8.48 release
4.8.49 rknn3_query
4.8.50 init_llm_session

1 Overview

RKNN3-Toolkit Lite provides Python interfaces for coprocessor model inference, making it easier for users to develop AI large model applications or validate models using Python.

1.1 Hardware Platforms

This document is applicable to the following hardware platforms:

- RK1820
- RK1828

1.2 Applicable system

- Debian: 10 (aarch64)
- Debian: 11 (aarch64)
- Debian: 12 (aarch64)

2 Development environment build

2.1 Requirements/Dependencies

RKNN3-Toolkit Lite requires the following environment:

Operating System	Debian 10 / 11 / 12 (aarch64)
Python Version	3.9 / 3.10 / 3.11 / 3.12
Python Libraries	'transformers', 'numpy', 'jinja2'

2.2 Installation

Use the `pip3 install` command to install RKNN3-Toolkit Lite. The installation steps are as follows:

If `python3/pip3` is not installed, use the following command to install it:

```
sudo apt-get update
sudo apt-get install -y python3 python3-dev python3-pip gcc
```

Note:

- Some dependent modules require source code compilation. In this case, `python3-dev` and `gcc` will be used. This step will install both packages simultaneously to avoid compilation failures during dependency installation.

Install the dependencies `opencv-python` and `numpy`:

```
sudo apt-get install -y python3-opencv
sudo apt-get install -y python3-numpy
```

or

```
pip3 install opencv-python
pip3 install numpy
```

Note:

- RKNN3-Toolkit Lite itself does not depend on `opencv-python`, but this module is needed for image processing in the examples.
- Installing `numpy` via `pip3` on Debian 10 firmware may fail. It is recommended to use the method described above.

Install RKNN3-Toolkit Lite:

All platform packages are located in the `rknn3-toolkit-lite/packages` directory of the SDK. Enter this directory and run the following commands to install RKNN3-Toolkit Lite:

```
# Python 3.9
pip3 install rknn3_toolkit_lite-x.y.z-cp39-cp39-linux_aarch64.whl
# Python 3.10
pip3 install rknn3_toolkit_lite-x.y.z-cp310-cp310-linux_aarch64.whl
# Python 3.11
pip3 install rknn3_toolkit_lite-x.y.z-cp311-cp311-linux_aarch64.whl
# Python 3.12
pip3 install rknn3_toolkit_lite-x.y.z-cp312-cp312-linux_aarch64.whl
```


3 Usage

RKNN3-Toolkit Lite is mainly used for deploying RKNN models on Rockchip NPU. Before using RKNN3-Toolkit Lite, users need to convert the models exported from various deep learning frameworks into RKNN models using RKNN3-Toolkit.

3.1 Basic Usage Process

The basic process for deploying RKNN models using RKNN3-Toolkit Lite is shown in the following figure:

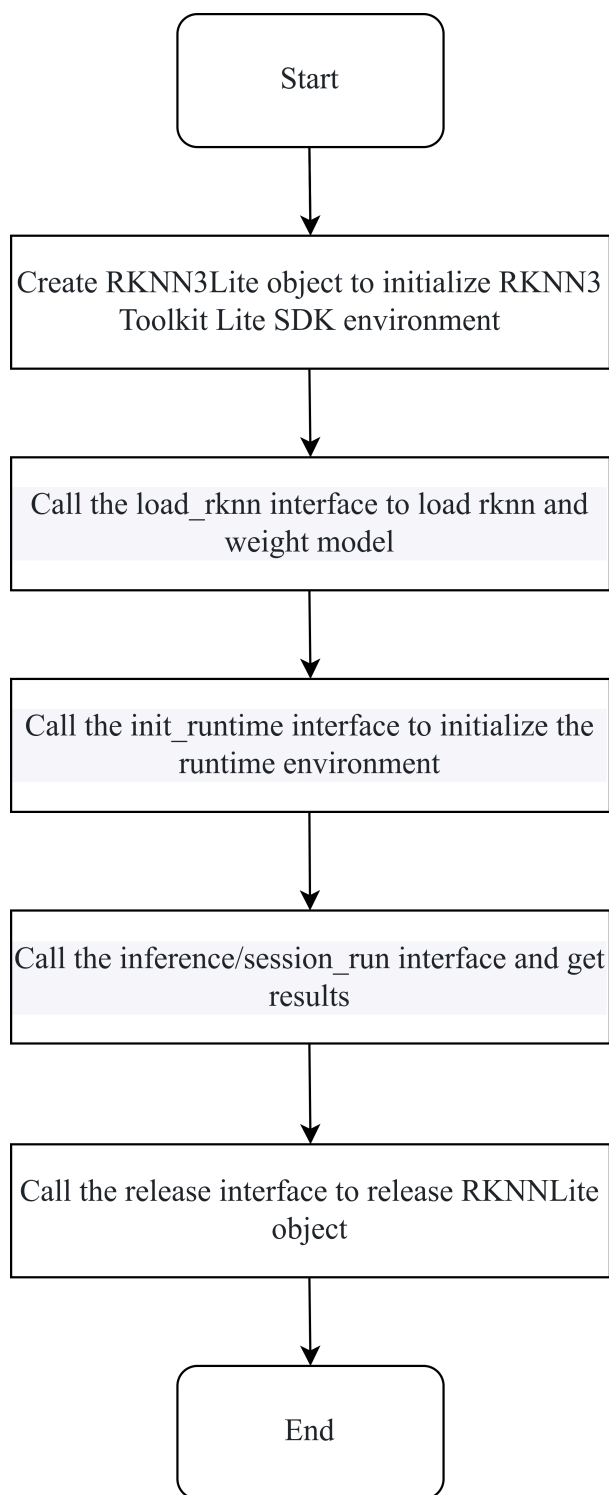


Figure 3-1 Basic usage flow of RKNN3-Toolkit Lite

Note:

1. Before calling the `inference` function, input data must be prepared and preprocessed. For pure LLMs, you need to obtain `input_ids`, and for multimodal models, you need to obtain `embeds`. Then, set the appropriate parameters for the `inference` function.
2. After calling the `inference` function, the inference result usually requires further processing to complete the application.

3.2 Example

An example VLM application based on RKNN3-Toolkit Lite is provided in the `SDK/rknn3-toolkit-lite/examples` directory. This application uses the RKNN3-Toolkit Lite interface to load the Qwen2.5VL-3B RKNN model and Tokenizer, performs inference on input images and prompts, and outputs the analysis results.

Run the example by following these steps:

1. Prepare a development board with RKNN3-Toolkit Lite installed.
2. Push the complete `SDK/rknn3-toolkit-lite/examples` directory to the board.
3. On the board, navigate to `examples/Qwen2.5VL` and run the example:

```
python test.py \  
  --rknn_vision_path XXX/Qwen2.5-VL-3B-vision.rknn \  
  --rknn_llm_path XXX/Qwen2.5-VL-3B-llm.rknn \  
  --tokenizer_path XXX/Tokenizer_Path \  
  --embed_path XXX/Qwen2.5-VL-3B-llm.embed.bin
```

Note: Please ensure that the specified RKNN model path matches the corresponding weight file path, as discrepancies may lead to inference failure.

Example output (partial):

```
--> Running vision model  
This image depicts a space scene. A large planet, possibly the Moon or Mars, is seen in the  
background, with craters and mountains. The sky is dotted with stars.  
  
In the foreground, an astronaut in a white suit is lying on the ground resting, holding a  
drink (perhaps beer), with a small box nearby. Other objects like a ladder, a stone, and  
various unidentified objects are scattered on the ground.  
  
The entire scene feels like a space exploration or lunar mission, but with a relaxed  
atmosphere as the astronaut appears to be taking a break and enjoying some relaxation time.  
  
-----Finished-----
```

Note: Due to differences in model versions, input images, or runtime environments, the actual output may slightly differ from the example above.

4 RKNN3 Type Definition

This chapter describes the type definitions, structures, enumerations, and constants used in RKNN3 Toolkit Lite. These definitions are based on `rknn3_types.py` and are used to describe RKNN3 API data structures and parameters.

4.1 Constants

RKNN3 Toolkit Lite defines the following constants:

- `RKNN3_MAX_DIMS = 16`: Maximum number of tensor dimensions
- `RKNN3_MAX_STRIDE_DIMS = RKNN3_MAX_DIMS + 1`: Maximum number of stride dimensions
- `RKNN3_MAX_NAME_LEN = 256`: Maximum name length
- `RKNN3_MAX_DYNAMIC_SHAPE_NUM = 512`: Maximum number of dynamic shapes
- `RKNN3_MAX_LORA_NUM = 32`: Maximum number of LoRA adapters
- `RKNN3_MAX_DEVS = 64`: Maximum number of devices
- `RKNN3_MAX_DEV_LEN = 64`: Maximum device ID length
- `RKNN3_MAX_NPU_NODE_NUM = 128`: Maximum number of NPU nodes
- `RKNN3_MAX_SPECIAL_BOS_ID_NUM = 64`: Maximum number of special BOS IDs
- `RKNN3_MAX_SPECIAL_EOS_ID_NUM = 64`: Maximum number of special EOS IDs
- `LIBRKNN3RT_PATH = '/usr/lib/librknn3_api.so'`: RKNN3 runtime shared library path

4.2 Enumerations

4.2.1 RKNN3QueryCmd

Query command types:

- `RKNN3_QUERY_IN_OUT_NUM = 0`: Query number of inputs/outputs
- `RKNN3_QUERY_INPUT_ATTR = 1`: Query input attributes
- `RKNN3_QUERY_OUTPUT_ATTR = 2`: Query output attributes
- `RKNN3_QUERY_PERF_DETAIL = 3`: Query performance details
- `RKNN3_QUERY_PERF_RUN = 4`: Query runtime performance
- `RKNN3_QUERY_SDK_VERSION = 5`: Query SDK version
- `RKNN3_QUERY_CORE_MEM_SIZE = 6`: Query core memory size
- `RKNN3_QUERY_CUSTOM_STRING = 7`: Query custom string
- `RKNN3_QUERY_ORIGIN_INPUT_ATTR = 8`: Query original input attributes
- `RKNN3_QUERY_ORIGIN_OUTPUT_ATTR = 9`: Query original output attributes
- `RKNN3_QUERY_NATIVE_INPUT_ATTR = RKNN3_QUERY_ORIGIN_INPUT_ATTR`: Query native input attributes, compatible with the old naming
- `RKNN3_QUERY_NATIVE_OUTPUT_ATTR = RKNN3_QUERY_ORIGIN_OUTPUT_ATTR`: Query native output attributes, compatible with the old naming

- `RKNN3_QUERY_NATIVE_NC1HWC2_INPUT_ATTR = RKNN3_QUERY_ORIGIN_INPUT_ATTR`: Query NC1HWC2 input attributes, compatible with the old naming
- `RKNN3_QUERY_NATIVE_NC1HWC2_OUTPUT_ATTR = RKNN3_QUERY_ORIGIN_OUTPUT_ATTR`: Query NC1HWC2 output attributes, compatible with the old naming
- `RKNN3_QUERY_NATIVE_NHWC_INPUT_ATTR = 10`: Query NHWC input attributes
- `RKNN3_QUERY_NATIVE_NHWC_OUTPUT_ATTR = 11`: Query NHWC output attributes
- `RKNN3_QUERY_DEVICE_MEM_INFO = 12`: Query device memory information
- `RKNN3_QUERY_CORE_NUMBER = 13`: Query number of cores
- `RKNN3_QUERY_ALLOCATION_INFO = 14`: Query allocation information
- `RKNN3_QUERY_DYNAMIC_SHAPE_CONFIG = 15`: Query dynamic shape configuration
- `RKNN3_QUERY_DYNAMIC_SHAPE_INFO = 16`: Query dynamic shape information
- `RKNN3_QUERY_LLM_CONFIG = 17`: Query LLM configuration
- `RKNN3_QUERY_POSTPROCESS_IN_OUT_NUM = 18`: Query number of post-process inputs/outputs
- `RKNN3_QUERY_POSTPROCESS_OUTPUT_ATTR = 19`: Query post-process output attributes
- `RKNN3_QUERY_POSTPROCESS_DYNAMIC_SHAPE_INFO = 20`: Query post-process dynamic shape information
- `RKNN3_QUERY_LORA_NUM = 21`: Query number of LoRA adapters
- `RKNN3_QUERY_LORA_INFO = 22`: Query LoRA information
- `RKNN3_QUERY_CMD_MAX = 23`: Maximum query command value

4.2.2 RKNN3TensorType

Tensor data types:

- `RKNN3_TENSOR_FLOAT32 = 0`: 32-bit float
- `RKNN3_TENSOR_FLOAT16 = 1`: 16-bit float
- `RKNN3_TENSOR_INT8 = 2`: 8-bit signed integer
- `RKNN3_TENSOR_UINT8 = 3`: 8-bit unsigned integer
- `RKNN3_TENSOR_INT16 = 4`: 16-bit signed integer
- `RKNN3_TENSOR_UINT16 = 5`: 16-bit unsigned integer
- `RKNN3_TENSOR_INT32 = 6`: 32-bit signed integer
- `RKNN3_TENSOR_UINT32 = 7`: 32-bit unsigned integer
- `RKNN3_TENSOR_INT64 = 8`: 64-bit signed integer
- `RKNN3_TENSOR_UINT64 = 9`: 64-bit unsigned integer
- `RKNN3_TENSOR_BOOL = 10`: Boolean
- `RKNN3_TENSOR_INT4 = 11`: 4-bit signed integer
- `RKNN3_TENSOR_FLOAT8E4M3FN = 12`: 8-bit floating-point format E4M3FN
- `RKNN3_TENSOR_BFLOAT16 = 13`: BF16 floating-point format
- `RKNN3_TENSOR_FLOAT8E8M0 = 14`: 8-bit floating-point format E8M0

- `RKNN3_TENSOR_FLOAT4E2M1 = 15` : 4-bit floating-point format E2M1
- `RKNN3_TENSOR_TYPE_MAX = 16` : Maximum tensor type value

4.2.3 RKNN3TensorQntType

Tensor quantization types:

- `RKNN3_TENSOR_QNT_NONE = 0` : None
- `RKNN3_TENSOR_PER_LAYER_SYMMETRIC = 1` : Per-layer symmetric
- `RKNN3_TENSOR_PER_LAYER_ASYMMETRIC = 2` : Per-layer asymmetric
- `RKNN3_TENSOR_PER_CHANNEL_SYMMETRIC = 3` : Per-channel symmetric
- `RKNN3_TENSOR_PER_CHANNEL_ASYMMETRIC = 4` : Per-channel asymmetric
- `RKNN3_TENSOR_PER_GROUP_SYMMETRIC = 5` : Per-group symmetric
- `RKNN3_TENSOR_PER_GROUP_ASYMMETRIC = 6` : Per-group asymmetric

4.2.4 RKNN3TensorLayout

Tensor layout formats:

- `RKNN3_TENSOR_UNDEFINED = 0` : Undefined
- `RKNN3_TENSOR_NCHW = 1` : NCHW
- `RKNN3_TENSOR_NHWC = 2` : NHWC
- `RKNN3_TENSOR_NC1HWC2 = 3` : NC1HWC2
- `RKNN3_TENSOR_CHWN = 4` : CHWN
- `RKNN3_TENSOR_HWIO = 5` : HWIO
- `RKNN3_TENSOR_OIHW = 6` : OIHW
- `RKNN3_TENSOR_O1I1HWI2O2 = 7` : O1I1HWI2O2

4.2.5 RKNN3MemAllocFlags

Memory allocation flags:

- `RKNN3_FLAG_MEMORY_FLAGS_DEFAULT = 0` : Default
- `RKNN3_FLAG_MEMORY_CACHEABLE = 1` : Cacheable
- `RKNN3_FLAG_MEMORY_NON_CACHEABLE = 2` : Non-cacheable

4.2.6 RKNN3MemSyncMode

Memory synchronization modes:

- `RKNN3_MEMORY_SYNC_TO_DEVICE = 1` : Sync to device
- `RKNN3_MEMORY_SYNC_FROM_DEVICE = 2` : Sync from device
- `RKNN3_MEMORY_SYNC_BIDIRECTIONAL = 3` : Bidirectional

4.2.7 RKNN3CoreMask

NPU core masks:

- `RKNN3_NPU_CORE_AUTO = 0`: Auto select
- `RKNN3_NPU_CORE_0 = 1`: Core 0
- `RKNN3_NPU_CORE_1 = 2`: Core 1
- `RKNN3_NPU_CORE_2 = 4`: Core 2
- `RKNN3_NPU_CORE_3 = 8`: Core 3
- `RKNN3_NPU_CORE_4 = 16`: Core 4
- `RKNN3_NPU_CORE_5 = 32`: Core 5
- `RKNN3_NPU_CORE_6 = 64`: Core 6
- `RKNN3_NPU_CORE_7 = 128`: Core 7
- `RKNN3_NPU_CORE_ALL = 0xFFFFFFFF`: All cores

4.2.8 RKNN3KVCachePolicy

KV-cache policies:

- `RKNN3_KVCACHE_POLICY_DEFAULT = 0`: Default
- `RKNN3_KVCACHE_POLICY_RECURRENT = 1`: Recurrent
- `RKNN3_KVCACHE_POLICY_NORMAL = 2`: Normal

4.2.9 RKNN3KVCacheClearPolicy

KV-cache clear policies:

- `RKNN3_KVCACHE_CLEAR_ALL = 0`: Clear all
- `RKNN3_KVCACHE_KEEP_SYSTEM_PROMPT = 1`: Keep system prompt

4.2.10 RKNN3LLMInputType

LLM input types:

- `RKNN3_LLM_INPUT_PROMPT = 0`: Prompt
- `RKNN3_LLM_INPUT_TOKEN = 1`: Token
- `RKNN3_LLM_INPUT_EMBED = 2`: Embed
- `RKNN3_LLM_INPUT_MULTIMODAL = 3`: Multimodal
- `RKNN3_LLM_INPUT_AUX = 4`: Auxiliary

4.2.11 LLMCallState

LLM callback states:

- `RKLLM_RUN_NORMAL = 0`: Running
- `RKLLM_RUN_WAITING = 1`: Waiting
- `RKLLM_RUN_FINISH = 2`: Finished

- `RKLLM_RUN_STOP = 3`: Stopped
- `RKLLM_RUN_MAX_NEW_TOKEN_REACHED = 4`: Max new tokens reached
- `RKLLM_RUN_ERROR = 5`: Error
- `RKLLM_RUN_PAUSE = 6`: Paused
- `RKLLM_RUN_RESUME = 7`: Resumed

4.2.12 Other Enumerations

In addition to the enumerations above, the current version also defines the following enumeration types:

- `RKNN3MemType`: `RKNN3_MEMORY_TYPE_NPU_DRAM = 0 << 0`, `RKNN3_MEMORY_TYPE_EXT_DDR = 1 << 0`
- `RKNN3KVCacheDataType`: KV Cache data type, including `UNDEFINED`, `INT4_TO_F16`, `INT4_TO_F8`, `INT8_TO_F16`, `FLOAT4_TO_F16`, `FLOAT4_TO_F8`, `FLOAT8_TO_F16`, `FLOAT8_TO_F8`, and `FLOAT16`
- `RKNN3KVCacheStoreMethod`: KV Cache storage method, including `UNDEFINED`, `NORMAL`, and `GROUP_QUANT`
- `LLMOutputCallbackState`: Output callback stage, including `PREFILL_PROCESSING`, `PREFILL_FINISHED`, `DECODE_PROCESSING`, and `DECODE_FINISHED`
- `RKNN3LLMTaskType`: LLM task type, including `RKNN3_LLM_TASK_GENERATE = 0` and `RKNN3_LLM_TASK_EMBEDDING = 1`
- `RKNN3ImFmt`: Image format, including `RGB888`, `BGR888`, `GRAY8`, `YCbCr/YCrCb 420/422 SP`, and `UNKNOWN`
- `RKNN3ImProcFlag`: Image processing flags, including `CROP`, `RESIZE`, `FILL`, `COLOR_SPACE_CONVERT`, `DECODE`, and `ENCODE`
- `RKNN3OpTargetType`: Custom operator target type, currently including `RKNN3_OP_TARGET_TYPE_CPU = 1`
- `RKNN3OpPluginType`: Custom operator plugin type, including `POSTPROCESS` and `CUSTOM_OP`

4.3 Basic Structures

4.3.1 RKNN3InputOutputNum

Input/output number information:

- `n_input`: Number of inputs (`uint32`)
- `n_output`: Number of outputs (`uint32`)

4.3.2 RKNN3QuantInfo

Quantization information:

- `scale`: Scale factor (`float`)
- `zero_point`: Zero point (`int32`)

4.3.3 RKNN3TensorAttr

Tensor attribute information:

- `index`: Index (`uint32`)
- `name`: Name (`char[256]`)

- `n_dims`: Number of dimensions (`uint32`)
- `shape`: Shape (`uint32[16]`)
- `aligned_size`: Aligned size (`uint64`)
- `n_stride`: Number of stride dimensions (`uint32`)
- `stride`: Stride (`uint64[17]`)
- `n_elems`: Number of elements (`uint32`)
- `dtype`: Data type (`int`)
- `layout`: Layout (`int`)
- `qnt_type`: Quantization type (`int`)
- `qnt_info`: Quantization info (`RKNN3QuantInfo`)
- `core_id`: Core ID (`int32`)

4.3.4 RKNN3PerfDetail

Performance detail:

- `perf_data`: Performance data (`char*`)
- `data_len`: Data length (`uint64`)

4.3.5 RKNN3PerfRun

Runtime performance:

- `run_duration`: Run duration (`int64`)

4.3.6 RKNN3SDKVersion

SDK version:

- `api_version`: API version (`char[256]`)
- `drv_version`: Driver version (`char[256]`)

4.3.7 RKNN3MemSize

Memory sizes:

- `total_const_size`: Total constant size (`uint64`)
- `total_internal_size`: Total internal size (`uint64`)
- `total_dma_allocated_size`: Total DMA allocated size (`uint64`)
- `total_sram_size`: Total SRAM size (`uint64`)
- `free_sram_size`: Free SRAM size (`uint64`)

4.3.8 RKNN3CustomString

Custom string:

- `string`: String (`char[1024]`)

4.3.9 RKNN3TensorMemory

Tensor memory:

- `virt_addr`: Virtual address (`void*`)
- `phys_addr`: Physical address (`uint64`)
- `fd`: File descriptor (`int32`)
- `buffer_id`: Buffer ID (`uint64`)
- `offset`: Offset (`uint64`)
- `size`: Size (`uint64`)
- `flags`: Flags (`uint64`)
- `core_id`: Core ID (`int32`)
- `priv_data`: Private data (`void*`)
- `mem_type`: Memory type (`int`)

4.3.10 RKNN3Config

RKNN3 configuration:

- `priority`: Priority (`int32`)
- `run_timeout`: Run timeout (`uint32`)
- `run_core_mask`: Core mask (`uint32`)
- `user_mem_weight`: User memory for weights (`uint8`)
- `user_mem_internal`: User internal memory (`uint8`)
- `user_sram`: User SRAM (`uint8`)

4.3.11 RKNN3Tensor

RKNN3 tensor:

- `mem`: Memory (`RKNN3TensorMemory*`)
- `attr`: Attributes (`RKNN3TensorAttr*`)

4.3.12 RKNN3AllocationInfo

Allocation information:

- `core_id`: Core ID (`int32`)
- `command_mem`: Command memory (`RKNN3TensorMemory`)
- `weight_mem`: Weight memory (`RKNN3TensorMemory`)
- `internal_mem`: Internal memory (`RKNN3TensorMemory`)

- `kvcache_mem`: KV-cache memory (`RKNN3TensorMemory`)
- `reserved`: Reserved (`uint8[128]`)

4.3.13 RKNN3ShapeInfo

Shape information:

- `shape_id`: Shape ID (`int32`)
- `n_inputs`: Number of inputs (`uint32`)
- `input_attrs`: Input attributes (`RKNN3TensorAttr*`)
- `n_outputs`: Number of outputs (`uint32`)
- `output_attrs`: Output attributes (`RKNN3TensorAttr*`)
- `is_default`: Is default (`uint8`)

4.3.14 RKNN3ShapeConfig

Shape configuration:

- `n_shapes`: Number of shapes (`uint32`)
- `current_shape_id`: Current shape ID (`int32`)

4.3.15 RKNN3InitExtend

Extended init parameters:

- `device_id`: Device ID (`char*`)
- `reserved`: Reserved (`uint8[128]`)

4.3.16 RKNN3NodeMemInfo

Node memory information:

- `total`: Total memory (`uint64`)
- `free`: Free memory (`uint64`)

4.3.17 RKNN3DevMemInfo

Device memory information:

- `node_num`: Number of nodes (`uint32`)
- `sys_total`: System total (`uint64`)
- `sys_free`: System free (`uint64`)
- `node_mem_info`: Node memory info (`RKNN3NodeMemInfo[128]`)

4.3.18 RKNN3Device

Device information:

- `id`: Device ID (`char[64]`)
- `type`: Device type (`char[64]`)
- `mem_info`: Memory info (`RKNN3DevMemInfo`)

4.3.19 RKNN3Devices

Device list:

- `n_devices`: Number of devices (`uint32`)
- `devices`: Device info (`RKNN3Device[64]`)

4.3.20 Other Structures

In addition to the structures above, the current version also defines the following structures for core memory query, LLM configuration, image memory, custom operators, and other scenarios:

- `RKNN3CoreMemSize`: Core memory size information. Fields include `core_id`, `weight_size`, `internal_size`, and `reserved`.
- `RKNN3LLMConfig`: LLM model configuration. Fields include `chat_template`, `vocab_size`, `embedding_dim`, `max_ctx_len`, `max_position_embeddings`, `kvcache_store_method`, `kvcache_dtype`, `kvcache_group_size`, `kvcache_residual_depth`, `model_type`, `task_type`, `rope_cache_host_storage`, and `reserved`.
- `RKNN3ImRect`: Image rectangle region. Fields include `x`, `y`, `width`, and `height`.
- `RKNN3ImMetadata`: Image metadata. Fields include `peer_im_mem_addr`.
- `RKNN3ImMem`: Image memory. Fields include `width`, `height`, `stride`, `format`, `sync_to_host`, `data_mem`, and `metadata`.
- `RKNN3CustomOpContext`: Custom operator context. Fields include `rknn_ctx`, `priv_data`, and `user_data`.
- `RKNN3CustomOp`: Custom operator registration information. Fields include `op_type`, `plugin_type`, `target`, `version`, `author`, `description`, and callbacks such as `init/prepare/compute/deinit/get_output_num/get_attrs`.
- `LLMInputCallbackParam`: LLM input callback parameters. Fields include `tokens`, `num_tokens`, `shape_id`, `pos`, and `reserved`.

4.4 LLM Related Structures

4.4.1 Float16

16-bit floating-point representation:

- `frac`: Fraction (`uint16`, 10 bits)
- `exp`: Exponent (`uint16`, 5 bits)
- `sign`: Sign bit (`uint16`, 1 bit)

4.4.2 RKNN3VocabInfo

Vocabulary information:

- `vocab_size`: Vocabulary size (`int`)
- `special_bos_id`: Special BOS IDs (`int[64]`)
- `special_eos_id`: Special EOS IDs (`int[64]`)
- `n_special_bos_id`: Number of special BOS IDs (`int`)
- `n_special_eos_id`: Number of special EOS IDs (`int`)
- `linefeed_id`: Linefeed token ID (`int`)
- `skip_special_token`: Skip special tokens (`bool`)
- `ignore_eos_token`: Ignore EOS token (`bool`)
- `reserved`: Reserved (`uint8[64]`)

4.4.3 RKNN3SamplingParams

Sampling parameters:

- `top_k`: Top-k (`int32`)
- `top_p`: Top-p (`float`)
- `temperature`: Temperature (`float`)
- `repeat_penalty`: Repeat penalty (`float`)
- `frequency_penalty`: Frequency penalty (`float`)
- `presence_penalty`: Presence penalty (`float`)

4.4.4 RECURRENT

Recurrent KV-cache parameters:

- `n_keep`: Keep count (`int64`)
- `n_keep_aligned`: Aligned keep count (`int64`)

4.4.5 RKNN3KVCachePolicyParam

KV-cache policy parameters:

- `recurrent`: Recurrent params (`RECURRENT*`)
- `reserved`: Reserved (`uint8[64]`)

4.4.6 RKNN3Lora (Not currently supported)

LoRA configuration:

- `lora_name`: LoRA name (`char*`)
- `scale`: Scale factor (`float`)

4.4.7 RKNN3LLMTensor

LLM input tensor:

- `name`: Name (char*)
- `prompt`: Prompt (char*)
- `embed`: Embedding (Float16*)
- `tokens`: Tokens (int32*)
- `n_tokens`: Number of tokens (uint64)
- `enable_thinking`: Enable thinking (bool)

4.4.8 RKNN3AuxTensor

Auxiliary tensor (same as `RKNN3Tensor`).

4.4.9 RKNN3Image

Image input:

- `image_embed`: Image embedding (Float16*)
- `n_image_tokens`: Number of image tokens (uint64)
- `n_image`: Number of images (uint64)
- `image_start`: Image start token string (char*)
- `image_end`: Image end token string (char*)
- `image_content`: Image content token string (char*)
- `image_width`: Image width (uint64)
- `image_height`: Image height (uint64)

4.4.10 RKNN3Audio

Audio input:

- `audio_embed`: Audio embedding (Float16*)
- `n_audio_tokens`: Number of audio tokens (uint64)
- `n_audio`: Number of audio clips (uint64)
- `audio_start`: Audio start token string (char*)
- `audio_end`: Audio end token string (char*)
- `audio_content`: Audio content token string (char*)

4.4.11 RKNN3Video

Video input:

- `video_embed`: Video embedding (Float16*)
- `n_frame_tokens`: Number of frame tokens (uint64)
- `n_frame_per_video`: Frames per video (uint64)

- `n_video`: Number of videos (`uint64`)
- `video_start`: Video start token string (`char*`)
- `video_end`: Video end token string (`char*`)
- `video_content`: Video content token string (`char*`)
- `frame_width`: Frame width (`uint64`)
- `frame_height`: Frame height (`uint64`)

4.4.12 RKNN3LLMMultiModelTensor

Multimodal LLM input tensor:

- `name`: Name (`char*`)
- `prompt`: Prompt (`char*`)
- `tokens`: Tokens (`int32*`)
- `n_tokens`: Number of tokens (`uint64`)
- `enable_thinking`: Enable thinking (`bool`)
- `image`: Image data (`RKNN3Image`)
- `audio`: Audio data (`RKNN3Audio`)
- `video`: Video data (`RKNN3Video`)

4.4.13 RKNN3LLMInputUnion

LLM input union:

- `llm_input`: LLM input (`RKNN3LLMTensor`)
- `multimodal_input`: Multimodal input (`RKNN3LLMMultiModelTensor`)
- `aux_input`: Auxiliary input (`RKNN3AuxTensor`)

4.4.14 RKNN3LLMInput

LLM input:

- `role`: Role (`char*`)
- `input_type`: Input type (`int`)
- `input_union`: Input union (`RKNN3LLMInputUnion`)

4.4.15 RKNN3LLMExtendParam

Extended LLM parameters:

- `reserved`: Reserved (`uint8[128]`)

4.4.16 RKNN3LLMParam

LLM parameters:

- `logits_name`: Logits tensor name (`char*`)
- `max_context_len`: Maximum context length (`int32`)
- `sampling_param`: Sampling params (`RKNN3SamplingParams`)
- `vocab_info`: Vocabulary info (`RKNN3VocabInfo`)
- `extend_param`: Extended params (`RKNN3LLMExtendParam`)

4.4.17 RKNN3LLMInferParam

LLM inference parameters:

- `keep_history`: Keep history (`int`)
- `max_new_tokens`: Max new tokens (`int32`)
- `reserved`: Reserved (`uint8[128]`)

4.4.18 RKLLMResult

LLM generation result:

- `token_ids`: Token IDs (`int*`)
- `num_tokens`: Number of tokens (`int`)

4.4.19 RKLLMRunState

LLM runtime state:

- `n_total_tokens`: Total tokens (`uint64`)
- `n_max_tokens`: Max tokens (`uint64`)
- `n_decode_tokens`: Decode tokens (`uint64`)
- `n_prefill_tokens`: Prefill tokens (`uint64`)
- `kvcache_policy`: KV-cache policy (`int`)
- `n_loras_enabled`: Enabled LoRA count (`int32`)
- `loras_enabled`: Enabled LoRA list (`RKNN3Lora*`)

4.4.20 RKLLMResultLastHiddenLayer

Last hidden-layer result:

- `hidden_states`: Hidden states (`float*`)
- `embd_size`: Embedding size (`int`)
- `num_tokens`: Number of tokens (`int`)

4.5 Callback Types

4.5.1 LLMResultCallback

```
int (*LLMResultCallback)(void* userdata, RKLLMResult* result, int state)
```

4.5.2 LLMSamplingCallback

```
int (*LLMSamplingCallback)(void* userdata, Float16* logits, char* token_str)
```

4.5.3 LLMGetEmbedCallback

```
int (*LLMGetEmbedCallback)(void* userdata, int32_t* tokens, uint64_t n_tokens, void* embeds, uint64_t embd_size)
```

4.5.4 LLMTokenizerCallback

```
int (*LLMTokenizerCallback)(void* userdata, char* text, int32_t text_len, int32_t* tokens, int32_t max_tokens)
```

4.5.5 LLMOutputCallback

```
int (*LLMOutputCallback)(void* userdata, RKNN3Tensor* tensors, uint32_t n_tensors, int state)
```

4.5.6 LLMInputCallback

```
int (*LLMInputCallback)(void* userdata, RKNN3Tensor* tensors, uint32_t n_tensors, LLMInputCallbackParam param)
```

The input callback is used by the LLM session to supply or update input tensors during runtime. The `LLMInputCallbackParam` argument provides the current token buffer, token count, shape ID, and position information. When this callback is used, configure `input_tensors_index` and `n_input_tensors` in `RKLLMCallback` to describe which model inputs are handled by the callback.

4.5.7 RKLLMCallback

LLM callback container. The current version supports six callback function entries:

- `result_callback`: `LLMResultCallback`
- `result_userdata`: user data (`void*`)
- `sampling_callback`: `LLMSamplingCallback`
- `sampling_userdata`: user data (`void*`)
- `tokenizer_callback`: `LLMTokenizerCallback`
- `tokenizer_userdata`: user data (`void*`)
- `embed_callback`: `LLMGetEmbedCallback`

- `embed_userdata`: user data (`void*`)
- `output_callback`: `LLMOutputCallback`
- `output_userdata`: user data (`void*`)
- `output_tensors`: output tensors (`RKNN3Tensor*`)
- `n_output_tensors`: number of output tensors (`uint32`)
- `input_callback`: input callback (`LLMInputCallback`)
- `input_userdata`: input callback user data (`void*`)
- `input_tensors_index`: input tensor indices (`int32*`)
- `n_input_tensors`: number of input tensors (`uint32`)

4.6 Wrapper Classes

4.6.1 RKNN3AuxTensorWrapper

Auxiliary tensor wrapper:

- `aux_data`: Auxiliary data (`numpy.ndarray`)
- `index`: Index (`int`)
- `align_size`: Aligned size (`int`)

4.7 Utility Functions

4.7.1 rknn_dtype_to_numpy_dtype

Description: Convert an RKNN3 tensor data type enum to the corresponding NumPy dtype.

Parameters:

- `dtype_enum`: RKNN3 tensor type enum value (`RKNN3TensorType`)

Returns: A NumPy dtype (`numpy.dtype`). If no mapping is found, `numpy.float32` is returned.

Example:

```
from rknn3lite.api.rknn3_types import rknn_dtype_to_numpy_dtype, RKNN3TensorType

np_dtype = rknn_dtype_to_numpy_dtype(RKNN3TensorType.RKNN3_TENSOR_FLOAT16)
print(np_dtype) # <class 'numpy.float16'>
```

4.7.2 rknn3_get_layout_string

Description: Get the string representation of a tensor layout enum. Useful for debugging/logs.

Parameters:

- `layout`: Tensor layout enum value (`RKNN3TensorLayout`)

Returns: Layout string (e.g. `"NCHW"`, `"NHWC"`). Returns `"UNKNOWN"` if invalid.

Example:

```
from rknn3lite.api.rknn3_types import rknn3_get_layout_string, RKNN3TensorLayout

layout_str = rknn3_get_layout_string(RKNN3TensorLayout.RKNN3_TENSOR_NHWC)
print(layout_str) # NHWC
```

4.7.3 rknn3_get_type_string

Description: Get the string representation of a tensor dtype enum (e.g. FP32/INT8). Useful for debugging/logs.

Parameters:

- `dtype`: Tensor dtype enum value (`RKNN3TensorType`)

Returns: Type string. Returns `"UNKNOWN"` if invalid.

Example:

```
from rknn3lite.api.rknn3_types import rknn3_get_type_string, RKNN3TensorType

type_str = rknn3_get_type_string(RKNN3TensorType.RKNN3_TENSOR_INT8)
print(type_str) # INT8
```

4.7.4 rknn3_get_qnt_type_string

Description: Get the string representation of a tensor quantization type enum, such as `NONE`, `PER_LAYER_SYMMETRIC`, or `PER_CHANNEL_ASYMMETRIC`. Returns `UNKNOWN` if the enum value is invalid.

Parameters:

- `qnt_type`: RKNN3 tensor quantization type enum value (`RKNN3TensorQntType`)

Returns: Quantization type string.

4.7.5 dump_tensor_attr

Description: Print detailed tensor attribute information to the console for debugging.

Parameters:

- `attr`: Tensor attribute (`RKNN3TensorAttr`)
- `prefix`: Prefix string, default `"tensor"`

Returns: None

Example:

```
from rknn3lite.api.rknn3_types import dump_tensor_attr

dump_tensor_attr(attr, prefix="input")
```

4.7.6 get_os_platform

Description: Get a platform string for the current OS/architecture, used for compatibility checks.

Parameters: None

Returns: Platform string, e.g. "Linux_x64" or "Windows_x32".

Example:

```
from rknn3lite.api.rknn3_types import get_os_platform

platform = get_os_platform()
print(platform)
```

4.8 RKNN3Lite Class

`RKNN3Lite` is the main class in RKNN3 Toolkit Lite. It provides model loading, runtime initialization, inference, LoRA, KV-cache management, memory management, asynchronous inference, debugging, and querying.

4.8.1 init

Description: Create an `RKNN3Lite` instance and configure logging.

Parameters:

- `llm_mode`: `bool`, whether to enable LLM mode. Default: `False`.
- `verbose`: `bool`, whether to print verbose logs to the terminal. Default: `False`.
- `verbose_file`: `str`, log file path. If provided, logs are also written to this file. Default: `None`.

Returns: None

Example:

```
from rknn3lite.api.rknn3_lite import RKNN3Lite

rknn_lite = RKNN3Lite(verbose=True, verbose_file='./inference.log')
rknn_lite = RKNN3Lite(verbose=True)
rknn_lite = RKNN3Lite(llm_mode=True, verbose=True)
```

4.8.2 load_rknn

Description: Load an RKNN model file and its weight file.

Parameters:

- `model_path`: RKNN model file path
- `weight_path`: RKNN weight file path

Returns: `0` on success, `-1` on failure

Example:

```
ret = rknn_lite.load_rknn(model_path='model.rknn', weight_path='model.weight')
if ret != 0:
    print('Load model failed')
```

4.8.3 init_runtime

Description: Initialize runtime. Must be called before inference.

Parameters:

- `target`: Coprocessor target. Currently supports `rk1820` / `rk1828`.
- `core_mask`: NPU core mask. The coprocessor has 8 NPU cores; select enabled cores via bitmask (e.g. `0x0f`).
- `llm_args`: Dict/list of dicts for LLM configuration.
- `llm_callback`: LLM callbacks for streaming inference.
- `llm_embed_path`: Embedding/vocab path used to initialize `LLMGetEmbedCallback`.
- `device_id`: Device ID used to differentiate multiple devices.

Returns: `0` on success, `-1` on failure

Notes:

- LLM configuration keys supported by `llm_args` include:
 - `top_k`, `top_p`, `temperature`, `repeat_penalty`, `frequency_penalty`, `presence_penalty`
 - `vocab_size`, `special_bos_id`, `special_eos_id`, `linefeed_id`
 - `skip_special_token`, `ignore_eos_token`
 - `keep_history`, `max_new_tokens`
 - `logits_name`, `max_context_len`
- `llm_callback` contains up to six callbacks:
 - `LLMResultCallback`, `LLMSamplingCallback`, `LLMTokenizerCallback`, `LLMGetEmbedCallback`, `LLMOutputCallback`, `LLMInputCallback`
 - If `LLMInputCallback` is used, also set `input_tensors_index` and `n_input_tensors` in `RKLLMCallback`.
- `device_id` can be obtained via `get_devices_id()`.
- If you want to query model information first and then initialize the LLM session, you can keep `llm_args` and `llm_callback` empty during `init_runtime`, and later call `init_llm_session()`.

Example (classic model):

```
# Get device id
device_id = rknn_lite.get_devices_id()
# Init runtime
ret = rknn_lite.init_runtime(target='rk1820', core_mask=0x01, device_id=device_id[0])
if ret != 0:
    print('Init runtime environment failed')
    exit(ret)
```

Example (LLM):

```
ARGS = [{"max_new_tokens": 256, "top_k": 1, "repeat_penalty": 1.1, "special_eos_id":
151645}]
callback = RKLLMCallback()

ret = rknn_lite.init_runtime(target='rk1820', core_mask=0xff, llm_args=ARGS,
llm_callback=callback)
if ret != 0:
    print('Init runtime environment failed')
    exit(ret)
```

4.8.4 inference

Description: Run inference for classic models.

Parameters:

- `inputs`: List of `numpy.ndarray`
- `data_format`: List of strings. Supported: `'undefined'`, `'nhwc'`, `'nchw'`. `'nhwc'` / `'nchw'` only apply to 4D inputs. For non-4D inputs, use `'undefined'` or omit.

Returns: List of `numpy.ndarray` on success; `None` on failure.

Example:

```
outputs = rknn_lite.inference(inputs=[input_data], data_format=['nhwc'])
```

4.8.5 session_run

Description: Run LLM inference (supports streaming output).

Parameters:

- `inputs`: A list of inputs for multimodal models (e.g. `RKNN3Image`, `RKNN3Audio`, `RKNN3Video`, `RKNN3AuxTensor`).
- `prompt`: Text prompt.
- `embeds`: Input embeddings (`numpy.ndarray`) to bypass tokenization. Typically 3D: `(batch, seq_len, hidden_size)`. **prompt and embeds are mutually exclusive.**
- `keep_history`: Whether to keep the history context during inference, which may affect the performance. The type is bool, and the default value is None, which means using the default setting of the model.

- `max_new_tokens`: The maximum number of tokens to be generated during inference, which may affect the performance. The type is int, and the default value is None, which means using the default setting of the model.
- `enable_thinking`: Whether to enable thinking mode during inference, which may affect the performance. The type is bool, and the default value is False, which means not using thinking mode.

Returns: `ret` (0 success, -1 failure) and a stats list: `[n_decode_tokens, n_prefill_tokens, llm_start_time, llm_end_time]`.

Example (LLM):

```
prompt = "Introduce how an LLM works."
ret, [n_decode_tokens, n_prefill_tokens, llm_start_time, llm_end_time] =
rknn_llm.session_run(prompt=prompt)
if ret != 0:
    print('RKNN llm inference failed!')
    exit(ret)
```

Example (VLM):

```
# Convert vision model output to required format
outputs = np.float16(np.expand_dims(rknn_vision.inference(inputs=[feature])[0], 0))

prompt = "<image>Please describe the image"
inputs = []
llm_input = RKNN3Image()
llm_input.image_embed = outputs.ctypes.data_as(ctypes.POINTER(Float16))
llm_input.n_image_tokens = outputs.shape[1]
llm_input.n_image = outputs.shape[0]
llm_input.image_width = 392
llm_input.image_height = 392
llm_input.image_start = "<|vision_start|>".encode('utf-8')
llm_input.image_end = "<|vision_end|>".encode('utf-8')
llm_input.image_content = "<|image_pad|>".encode('utf-8')
inputs.append(llm_input)

ret, [n_decode_tokens, n_prefill_tokens, llm_start_time, llm_end_time] =
rknn_llm.session_run(inputs=inputs, prompt=prompt)
if ret != 0:
    print('RKNN LLM inference failed!')
    exit(ret)
```

4.8.6 get_sdk_version

Description: Get RKNN SDK version.

Parameters: None

Returns: Version string; `None` on failure

Example:

```
version = rknn_lite.get_sdk_version()
print(version)
```

4.8.7 set_chat_template

Description: Set a chat template for the LLM.

Parameters:

- `system_prompt`: System prompt
- `prompt_prefix`: Prefix before each user turn
- `prompt_postfix`: Postfix after each user turn

Returns: `0` on success, `-1` on failure

Example:

```
system_prompt = "<|im_start|>system\nYou are a helpful assistant.<|im_end|>\n"
prompt_prefix = "<|im_start|>user\n"
prompt_postfix = "<|im_end|>\n<|im_start|>assistant\n"

ret = rknn_lite.set_chat_template(system_prompt, prompt_prefix, prompt_postfix)
if ret != 0:
    print('Set chat template failed!')
    exit(ret)
```

4.8.8 get_devices_id

Description: Get a list of connected device IDs.

Parameters: None

Returns: A list of device IDs

Example:

```
devices = rknn_lite.get_devices_id()
print(devices)
```

Note: This API is useful for multi-device deployment. For single-device use, you can take the first element or pass `None` (as supported by `init_runtime`).

4.8.9 get_inputs_tensor_attr

Description: Get input tensor attributes.

Parameters: None

Returns: List of input tensor attributes; `None` on failure

Example:

```
attrs = rknn_lite.get_inputs_tensor_attr()
```

4.8.10 get_outputs_tensor_attr

Description: Get output tensor attributes.

Parameters: None

Returns: List of output tensor attributes; `None` on failure

Example:

```
attrs = rknn_lite.get_outputs_tensor_attr()
```

4.8.11 init_lora (Not currently supported)

Description: Initialize LoRA support and load LoRA weights.

Parameters:

- `lora_weight_path`: LoRA weight file path or directory
- `weight_data`: Optional in-memory LoRA weight data. Supports `bytes`/`bytearray`/`memoryview`, `numpy.ndarray`, or a raw pointer.
- `weight_size`: Data size in bytes when `weight_data` is a raw pointer.

Returns: `0` on success, `-1` on failure

Example:

```
ret = rknn_lite.init_lora(lora_weight_path='lora_weights.bin')
```

4.8.12 load_lora (Not currently supported)

Description: Load a LoRA adapter.

Parameters:

- `lora`: `RKNN3Lora` (contains `lora_name` and `scale`)

Returns: `0` on success, `-1` on failure

Example:

```
lora = RKNN3Lora(lora_name='adapter1', scale=1.0)
ret = rknn_lite.load_lora(lora)
```

4.8.13 unload_lora (Not currently supported)

Description: Unload a LoRA adapter.

Parameters:

- `lora`: `RKNN3Lora`

Returns: `0` on success, `-1` on failure

Example:


```
ret = rknn_lite.unload_lora(lora)
```

4.8.14 enable_lora (Not currently supported)

Description: Enable a LoRA adapter for inference.

Parameters:

- `lora`: `RKNN3Lora`

Returns: `0` on success, `-1` on failure

Example:

```
ret = rknn_lite.enable_lora(lora)
```

4.8.15 disable_lora (Not currently supported)

Description: Disable a LoRA adapter.

Parameters:

- `lora`: `RKNN3Lora`

Returns: `0` on success, `-1` on failure

Example:

```
ret = rknn_lite.disable_lora(lora)
```

4.8.16 query_lora (Not currently supported)

Description: Query loaded LoRA information.

Parameters: None

Returns: `(lora_list, n_lora)` on success; `(None, 0)` on failure.

Example:

```
lora_list, n_lora = rknn_lite.query_lora()
```

4.8.17 set_kvcache_policy

Description: Set KV-cache policy.

Parameters:

- `policy`: `RKNN3KVCachePolicy`
- `param`: `RKNN3KVCachePolicyParam` or `None`

Returns: `0` on success, `-1` on failure

Example:

```
ret = rknn_lite.set_kvcache_policy(RKNN3KVCachePolicy.RKNN3_KVCACHE_POLICY_DEFAULT)
```

4.8.18 load_kvcache

Description: Load KV-cache from a file or memory.

Parameters:

- `kvcache_path`: KV-cache file path
- `kvcache_data`: KV-cache data in memory
- `size`: Data size

Returns: `0` on success, `-1` on failure

Example:

```
ret = rknn_lite.load_kvcache(kvcache_path='cache.bin')
```

4.8.19 save_kvcache

Description: Save KV-cache to a file.

Parameters:

- `kvcache_path`: Output file path

Returns: `0` on success, `-1` on failure

Example:

```
ret = rknn_lite.save_kvcache('cache.bin')
```

4.8.20 clear_kvcache

Description: Clear KV-cache.

Parameters:

- `clear_policy`: `RKNN3KVCacheClearPolicy` or `None`

Returns: `0` on success, `-1` on failure

Example:

```
ret = rknn_lite.clear_kvcache()
```

4.8.21 session_run_async

Description: Run LLM inference asynchronously.

Parameters:

- `inputs`: Input list for multimodal models
- `prompt`: Prompt text

- `embeds`: Embeddings
- `keep_history`: Whether to keep history context
- `max_new_tokens`: Maximum number of newly generated tokens
- `enable_thinking`: Whether to enable thinking mode

Returns: `(ret, stats)`, where `stats` is `[n_decode_tokens, n_prefill_tokens, llm_start_time, llm_end_time]`

Example:

```
ret, stats = rknn_lite.session_run_async(prompt="Async prompt")
```

4.8.22 session_stop

Description: Stop a running LLM session.

Parameters: None

Returns: `0` on success, `-1` on failure

Example:

```
ret = rknn_lite.session_stop()
```

4.8.23 session_pause

Description: Pause a running LLM session.

Parameters: None

Returns: `0` on success, `-1` on failure

Example:

```
ret = rknn_lite.session_pause()
```

4.8.24 session_resume

Description: Resume a paused LLM session.

Parameters: None

Returns: `0` on success, `-1` on failure

Example:

```
ret = rknn_lite.session_resume()
```

4.8.25 session_query_state

Description: Query current state of the LLM session.

Parameters: None

Returns: `RKLLMRunState` on success; `None` on failure

Example:

```
state = rknn_lite.session_query_state()
```

4.8.26 set_function_tools

Description: Set function tools (function calling) for the LLM session.

Parameters:

- `tools`: Tool definition string in JSON format

Returns: `0` on success, `-1` on failure

Example:

```
ret = rknn_lite.set_function_tools('{"tools": []}')
```

4.8.27 set_llm_param

Description: Set LLM parameters for an initialized session.

Parameters:

- `params`: `RKNN3LLMParam`
- `n_params`: Number of params

Returns: `0` on success, `-1` on failure

Example:

```
ret = rknn_lite.set_llm_param(params, 1)
```

4.8.28 dup_context

Description: Duplicate the current RKNN3 context.

Parameters: None

Returns: `(ret, new_context)`

Example:

```
ret, new_ctx = rknn_lite.dup_context()
```

4.8.29 inference_async

Description: Run classic model inference asynchronously.

Parameters:

- `inputs`: Input list
- `data_format`: Data format list

Returns: `0` on success, `-1` on failure

Example:

```
ret = rknn_lite.inference_async(inputs=[data])
```

4.8.30 wait

Description: Wait for the asynchronous inference to complete.

Parameters: None

Returns: `0` on success, `-1` on failure

Example:

```
ret = rknn_lite.wait()
```

4.8.31 get_outputs

Description: Get outputs from asynchronous inference.

Parameters: None

Returns: Output list

Example:

```
outputs = rknn_lite.get_outputs()
```

4.8.32 create_mem

Description: Create and bind tensor memory.

Parameters:

- `attr`: `RKNN3TensorAttr`
- `tensor`: Optional `RKNN3Tensor`
- `flags`: Allocation flags

Returns: `RKNN3TensorMemory` on success; `None` on failure

Example:

```
mem = rknn_lite.create_mem(attr)
```

4.8.33 create_mem_from_phys

Description: Create tensor memory from a physical address.

Parameters:

- `phys_addr`: Physical address
- `virt_addr`: Virtual address
- `size`: Size

Returns: `RKNN3TensorMemory` or `None`

Example:

```
mem = rknn_lite.create_mem_from_phys(phys_addr, virt_addr, size)
```

4.8.34 create_mem_from_fd

Description: Create tensor memory from a file descriptor.

Parameters:

- `fd`: File descriptor
- `virt_addr`: Virtual address
- `size`: Size
- `offset`: Offset

Returns: `RKNN3TensorMemory` or `None`

Example:

```
mem = rknn_lite.create_mem_from_fd(fd, virt_addr, size)
```

4.8.35 destroy_mem

Description: Destroy tensor memory.

Parameters:

- `mem`: `RKNN3TensorMemory`

Returns: `0` on success, `-1` on failure

Example:

```
ret = rknn_lite.destroy_mem(mem)
```

4.8.36 mem_sync

Description: Synchronize tensor memory.

Parameters:

- `mem`: RKNN3TensorMemory
- `sync_mode`: Sync mode

Returns: 0 on success, -1 on failure

Example:

```
ret = rknn_lite.mem_sync(mem, sync_mode)
```

4.8.37 set_shape

Description: Set the shape ID for dynamic-shape models.

Parameters:

- `shape_id`: Shape ID

Returns: 0 on success, -1 on failure

Example:

```
ret = rknn_lite.set_shape(0)
```

4.8.38 set_kvcache_mem

Description: Set KV-cache memory for specified NPU cores.

Parameters:

- `mem_list`: List of RKNN3TensorMemory
- `npu_core_indices`: Core index list

Returns: 0 on success, -1 on failure

Example:

```
ret = rknn_lite.set_kvcache_mem(mem_list, [0, 1])
```

4.8.39 set_internal_mem

Description: Set user-allocated internal memory.

Parameters:

- `mem_list`: List of RKNN3TensorMemory

Returns: 0 on success, -1 on failure

Example:

```
ret = rknn_lite.set_internal_mem(mem_list)
```

4.8.40 dump_features

Description: Dump intermediate features to `.npy` files.

Parameters:

- `dump_dir`: Output directory

Returns: `0` on success, `-1` on failure

Example:

```
ret = rknn_lite.dump_features('./dump')
```

4.8.41 profile_ops

Description: Print per-op performance information.

Parameters:

- `log_level`: Log level

Returns: `0` on success, `-1` on failure

Example:

```
ret = rknn_lite.profile_ops(1)
```

4.8.42 profile_mem

Description: Print memory usage for each NPU core.

Parameters: None

Returns: `0` on success, `-1` on failure

Example:

```
ret = rknn_lite.profile_mem()
```

4.8.43 register_custom_ops_plugins

Description: Register custom op plugins.

Parameters:

- `plugin_path`: Plugin shared library path

Returns: `0` on success, `-1` on failure

Example:

```
ret = rknn_lite.register_custom_ops_plugins('./plugin.so')
```


4.8.44 im_mem_create

Description: Create image memory.

Parameters:

- `width`: Width
- `height`: Height
- `fmt`: Image format
- `size`: Size
- `core_id`: Core ID
- `flags`: Flags

Returns: (`ret`, `RKNN3ImMem`)

Example:

```
ret, im_mem = rknn_lite.im_mem_create(224, 224, fmt, 0, 0)
```

4.8.45 im_mem_destroy

Description: Destroy image memory.

Parameters:

- `im_mem`: `RKNN3ImMem`

Returns: `0` on success, `-1` on failure

Example:

```
ret = rknn_lite.im_mem_destroy(im_mem)
```

4.8.46 im_cvt_color

Description: Convert image color space.

Parameters:

- `src_im_mem`: Source image memory
- `dst_im_mem`: Destination image memory

Returns: `0` on success, `-1` on failure

Example:

```
ret = rknn_lite.im_cvt_color(src, dst)
```

4.8.47 create_output_tensors

Description: Create output tensors with allocated memory, used by the LLM output callback.

Parameters:

- `output_indices`: Output tensor indices

Returns: (`output_tensor_array`, `n_output_tensors`)

Example:

```
tensors, n = rknn_lite.create_output_tensors([0, 1])
```

4.8.48 release

Description: Release RKNN resources.

Parameters: None

Returns: None

Example:

```
rknn_lite.release()
```

4.8.49 rknn3_query

Description: Generic query API.

Parameters:

- `cmd`: `RKNN3QueryCmd`
- `index`: Index

Returns: Query result

Example:

```
result = rknn_lite.rknn3_query(RKNN3QueryCmd.RKNN3_QUERY_SDK_VERSION)
```

4.8.50 init_llm_session

Description: Initialize an LLM session separately.

In some workflows, you may want to query model information before starting an LLM session. In this case, call `init_llm_session()` after `init_runtime()`.

Parameters:

- `llm_args`: LLM args/config
- `llm_callback`: Callback container
- `llm_embed_path`: Embedding path

Returns: `0` on success, `-1` on failure

Recommended workflow:

1. `load_rknn(model_path, weight_path)`
2. `init_runtime(target, core_mask)` (do not set `llm_args` / `llm_callback`; performs `rknn3_init` + model load + model init)
3. `rknn3_query(...)` (query IO/core/memory, etc.)
4. `init_llm_session(llm_args, callback)` (performs `rknn3_session_init` + set callbacks)
5. `session_run(...)`

Example:

```
ret = rknn_lite.init_runtime(target='rk1820', core_mask=LLM_CORE_MASK)

# ... rknn3_query

ret = rknn_lite.init_llm_session(llm_args=ARGS, llm_callback=callback)
```